

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1401

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

Duration :60 Mins

On Class

Total Marks:50

QUESTIONS:5

Annexure A

DESIGN TASK

Code of Conduct:

I Dinesh V (192524309) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Dinesh V

Q1. Consider the grammar:

$S \rightarrow ABD$

$A \rightarrow a \mid Db \mid \epsilon$

$B \rightarrow gD \mid dA \mid \epsilon$

$D \rightarrow e \mid f$

A compiler development team is developing a table-driven predictive parser for a domain-specific language. Before implementation, the team must determine whether the grammar is suitable for LL(1) parsing and design all necessary parsing components.

Tasks

(i) Construct FIRST and FOLLOW Sets

Compute the FIRST and FOLLOW sets for all non-terminals (S, A, B, and D). Clearly show the steps involved in the computation.

(ii) Construct the Predictive Parsing Table

Using the computed FIRST and FOLLOW sets, design the LL(1) predictive parsing table for the grammar.

(iii) Verify the LL(1) Property

Analyze the predictive parsing table and determine whether the grammar satisfies the LL(1) condition.

Your analysis should include:

FIRST/FIRST conflict analysis

FIRST/FOLLOW conflict analysis for ϵ -productions

Identification of conflicting or multiple entries in the parsing table

Final conclusion on whether the grammar is LL(1)

(iv) Demonstrate Predictive Parsing

Using the constructed parsing table, perform predictive parsing for a valid input string derived from the grammar.

Your demonstration should include:

Stack contents at each step

Remaining input

Parsing action performed

Final acceptance or rejection of the input string

(v) Provide Design Recommendations

Based on your analysis:

State whether the grammar can be directly used for LL(1) parsing.

If the grammar is not LL(1), identify the cause of the conflict and suggest suitable grammar modifications.

Discuss the advantages of using a predictive parser for syntax analysis. (10 Marks)

Ans:

Grammar

$$\begin{aligned} S &\rightarrow ABD \\ A &\rightarrow a \mid Db \mid \epsilon \\ B &\rightarrow gD \mid dA \mid \epsilon \\ D &\rightarrow ef \end{aligned}$$

(i) Compute FIRST and FOLLOW Sets

FIRST Sets Computation:

1. $D \rightarrow ef$:

$$\text{FIRST}(D) = \{e\}$$

2. $A \rightarrow a \mid Db \mid \epsilon$:

- $A \rightarrow a \Rightarrow a \in \text{FIRST}(A)$
- $A \rightarrow Db \Rightarrow \text{FIRST}(D) \subseteq \text{FIRST}(A) \Rightarrow e \in \text{FIRST}(A)$
- $A \rightarrow \epsilon \Rightarrow \epsilon \in \text{FIRST}(A)$

$$\text{FIRST}(A) = \{a, e, \epsilon\}$$

3. $B \rightarrow gD \mid dA \mid \epsilon$:

- $B \rightarrow gD \Rightarrow g \in \text{FIRST}(B)$
- $B \rightarrow dA \Rightarrow d \in \text{FIRST}(B)$
- $B \rightarrow \epsilon \Rightarrow \epsilon \in \text{FIRST}(B)$

$$\text{FIRST}(B) = \{g, d, \epsilon\}$$

4. $S \rightarrow ABD$:

- $\text{FIRST}(S) = \text{FIRST}(ABD)$
- Since $e \in \text{FIRST}(D)$ and $\epsilon \notin \text{FIRST}(D)$, S must derive at least e .
- $\text{FIRST}(A) \setminus \{\epsilon\} = \{a, e\}$
- If $A \Rightarrow \epsilon$, then $\text{FIRST}(B) \setminus \{\epsilon\} = \{g, d\}$
- If both $A \Rightarrow \epsilon$ and $B \Rightarrow \epsilon$, then $\text{FIRST}(D) = \{e\}$

$$\text{FIRST}(S) = \{a, e, g, d\}$$

FOLLOW Sets Computation:

1. S :

- Start symbol:

$$\text{FOLLOW}(S) = \{\$ \}$$

2. D :

- From $S \rightarrow ABD$: $\text{FOLLOW}(S) \subseteq \text{FOLLOW}(D) \Rightarrow \$ \in \text{FOLLOW}(D)$
- From $A \rightarrow Db$: $b \in \text{FOLLOW}(D)$
- From $B \rightarrow gD$: $\text{FOLLOW}(B) \subseteq \text{FOLLOW}(D)$

$$\text{FOLLOW}(D) = \{b, \$ \} \cup \text{FOLLOW}(B)$$

3. B :

- From $S \rightarrow ABD$: $\text{FIRST}(D) \subseteq \text{FOLLOW}(B) \Rightarrow \{e\} \subseteq \text{FOLLOW}(B)$

$$\text{FOLLOW}(B) = \{e\}$$

- Updating $\text{FOLLOW}(D)$:

$$\text{FOLLOW}(D) = \{b, e, \$ \}$$

4. A :

- From $S \rightarrow ABD$: $\text{FIRST}(BD) \subseteq \text{FOLLOW}(A)$
- $\text{FIRST}(BD) = (\text{FIRST}(B) \setminus \{\epsilon\}) \cup \text{FIRST}(D) = \{g, d\} \cup \{e\} = \{d, e, g\}$
- Since B can derive ϵ , $\text{FIRST}(D) \subseteq \text{FOLLOW}(A)$ as well.

- From $B \rightarrow dA$: $\text{FOLLOW}(B) \subseteq \text{FOLLOW}(A) \Rightarrow e \in \text{FOLLOW}(A)$
 $\text{FOLLOW}(A) = \{d, e, g\}$

Summary of FIRST and FOLLOW Sets

FIRST and FOLLOW Sets Table

Non-Terminal	FIRST Set	FOLLOW Set
S	{a, d, e, g}	{ $\$$ }
A	{a, e, ϵ }	{d, e, g}
B	{d, g, ϵ }	{e}
D	{e}	{b, e, $\$$ }

(ii) Construct $LL(1)$ Predictive Parsing Table

To fill the parsing table $M[X, a]$:

- For production $X \rightarrow \alpha$, for each terminal $a \in \text{FIRST}(\alpha)$, add $X \rightarrow \alpha$ to $M[X, a]$.
- If $\epsilon \in \text{FIRST}(\alpha)$, for each terminal/symbol $b \in \text{FOLLOW}(X)$, add $X \rightarrow \alpha$ to $M[X, b]$.

Predictive Parsing Table for CSA14 Assessment

Non-Terminal	a	b	d	e	g	$\$$
S	$S \rightarrow ABD$		$S \rightarrow ABD$	$S \rightarrow ABD$	$S \rightarrow ABD$	
A	$A \rightarrow a$		$A \rightarrow \epsilon$	$A \rightarrow Db \mid A \rightarrow \epsilon$	$A \rightarrow \epsilon$	
B			$B \rightarrow dA$	$B \rightarrow \epsilon$	$B \rightarrow gD$	
D				$D \rightarrow ef$		

(iii) Verify $LL(1)$ Property

- FIRST/FIRST Conflict Analysis:**
 - For $A \rightarrow a$ and $A \rightarrow Db$:
 $\text{FIRST}(a) \cap \text{FIRST}(Db) = \{a\} \cap \{e\} = \emptyset$ (No conflict)
- FIRST/FOLLOW Conflict Analysis (ϵ -productions):**
 - For $A \rightarrow \epsilon$, check $\text{FIRST}(A \rightarrow Db) \cap \text{FOLLOW}(A)$:
 $\text{FIRST}(Db) \cap \text{FOLLOW}(A) = \{e\} \cap \{d, e, g\} = \{e\} \neq \emptyset$
 - Conflict detected!** Symbol e belongs to both $\text{FIRST}(A \rightarrow Db)$ and $\text{FOLLOW}(A)$.
- Multiple Entries in Parsing Table:**
 - Entry $M[A, e]$ contains two productions: $A \rightarrow Db$ and $A \rightarrow \epsilon$.
- Final Conclusion:**
 - The grammar is **NOT** $LL(1)$ due to a FIRST/FOLLOW conflict at entry $M[A, e]$.

(iv) Predictive Parsing Demonstration

Note: Since $M[A, e]$ has a conflict, we choose the production $A \rightarrow Db$ when encountering input terminal e to demonstrate a successful derivation for input string $a d e f e f$.

- Input string:** $a d e f e f \$$
- Target Derivation:** $S \Rightarrow ABD \Rightarrow aBD \Rightarrow adAD \Rightarrow ad\epsilon D \Rightarrow adefD \Rightarrow adefef$

Step	Stack	Remaining Input	Action / Production Used
1	\$S	a d e f e f \$	$S \rightarrow ABD$
2	\$DBA	a d e f e f \$	$A \rightarrow a$
3	\$DBa	a d e f e f \$	Match a
4	\$DB	d e f e f \$	$B \rightarrow dA$
5	\$DAd	d e f e f \$	Match d
6	\$DA	e f e f \$	$A \rightarrow \epsilon$ (resolving conflict)
7	\$D	e f e f \$	$D \rightarrow ef$
8	\$fe	e f e f \$	Match e
9	\$f	f e f \$	Match f
10	\$D	e f \$	$D \rightarrow ef$
11	\$fe	e f \$	Match e
12	\$f	f \$	Match f
13	\$	ACCEPT	

(v) Design Recommendations

1. **Direct Applicability:** The grammar **cannot** be directly used for an $LL(1)$ parser due to the multiple-entry collision at $M[A, e]$.
2. **Cause of Conflict & Grammar Modification:**
 - *Cause:* Non-terminal A can derive Db (starting with e) or derive ϵ , while e is also in $FOLLOW(A)$ via $B \rightarrow \epsilon$ followed by D .
 - *Modification:* Left-factor or rewrite rules for A and B to eliminate overlapping lookaheads for symbol e .
3. **Advantages of Predictive Parsing:**
 - Direct linear time complexity $\mathcal{O}(n)$ without backtracking.
 - Simple deterministic implementation using an explicit stack or recursive functions.
 - Clear, localized syntax error detection.

Question 2: Shift-Reduce Parsing with Operator Precedence

Grammar

$$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid (E) \mid \text{id}$$

- **Input string:** $\text{id} * \text{id} / (\text{id} + \text{id})$

Operator Precedence & Associativity Rules

To resolve Shift-Reduce conflicts deterministically:

- Precedence hierarchy: $() > *, / > +, -$
- Associativity: Left-associative for $+, -, *, /$

Detailed Parsing Trace

Step	Stack	Remaining Input	Action	Handle Recognized / Notes
1	\$	$\text{id} * \text{id} / (\text{id} + \text{id}) \$$	Shift	—
2	\$ id	$* \text{id} / (\text{id} + \text{id}) \$$	Reduce	Handle: $\text{id} (E \rightarrow \text{id})$
3	\$ E	$* \text{id} / (\text{id} + \text{id}) \$$	Shift	—
4	\$ E *	$\text{id} / (\text{id} + \text{id}) \$$	Shift	—
5	\$ E * id	$/ (\text{id} + \text{id}) \$$	Reduce	Handle: $\text{id} (E \rightarrow \text{id})$

Step	Stack	Remaining Input	Action	Handle Recognized / Notes
6	\$ E * E	/ (id + id) \$	Reduce	Handle: E * E ($E \rightarrow E * E$, left-associative)
7	\$ E	/ (id + id) \$	Shift	—
8	\$ E /	(id + id) \$	Shift	—
9	\$ E / (	id + id) \$	Shift	—
10	\$ E / (id	+ id) \$	Reduce	Handle: id ($E \rightarrow id$)
11	\$ E / (E	+ id) \$	Shift	—
12	\$ E / (E +	id) \$	Shift	—
13	\$ E / (E + id	) \$	Reduce	Handle: id ($E \rightarrow id$)
14	\$ E / (E + E	) \$	Reduce	Handle: E + E ($E \rightarrow E + E$)
15	\$ E / (E	) \$	Shift	—
16	\$ E / (E)	\$	Reduce	Handle: (E) ($E \rightarrow (E)$)
17	\$ E / E	\$	Reduce	Handle: E / E ($E \rightarrow E / E$)
18	\$ E	\$	ACCEPT	Input successfully parsed

Conflict Resolution Analysis

- **Shift/Reduce Conflict at Step 6 (\$ E * E with input /):**
 - *Conflict:* Should the parser reduce $E * E$ or shift $/$?
 - *Resolution:* Both $*$ and $/$ share equal precedence and left associativity. The leftmost operator $*$ must be reduced first.
- **Shift/Reduce Conflict inside Parentheses (Step 11 & 14):**
 - Operators inside parentheses take priority over external operators due to higher precedence of $()$.

Q2. Consider the grammar:

$E \rightarrow E + E \mid E - E$

$E \rightarrow E * E \mid E / E$

$E \rightarrow (E)$

$E \rightarrow id$

A compiler development team is implementing a stack-based shift-reduce parser for arithmetic expressions.

To validate the parser design, the team must analyze the parsing process for the following input string: $id * id / (id + id)$

Your Design Should:

(i). Construct the stack implementation of the shift-reduce parser for the given grammar.

(ii). Demonstrate the complete parsing process by showing all shift and reduce operations performed on the input string.

- (iii). Identify the handle recognized at each reduction step and specify the corresponding production used.
- (iv). Explain the parser's decisions during parsing and discuss any shift-reduce conflicts that may arise, along with the role of operator precedence and associativity in resolving them.
- (v). Present a detailed parsing trace showing:

Stack contents

Remaining input

Parsing action (Shift / Reduce / Accept)

- (vi). Verify whether the input string is successfully accepted by the parser. (10 Marks)

Grammar

$$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid (E) \mid \text{id}$$

- **Input string:** $\text{id} * \text{id} / (\text{id} + \text{id})$

Operator Precedence & Associativity Rules

To resolve Shift-Reduce conflicts deterministically:

- Precedence hierarchy: $() > *, / > +, -$
- Associativity: Left-associative for $+, -, *, /$

Detailed Parsing Trace

Step	Stack	Remaining Input	Action	Handle Recognized / Notes
1	\$	$\text{id} * \text{id} / (\text{id} + \text{id}) \$$	Shift	—
2	\$ id	$* \text{id} / (\text{id} + \text{id}) \$$	Reduce	Handle: id ($E \rightarrow \text{id}$)
3	\$ E	$* \text{id} / (\text{id} + \text{id}) \$$	Shift	—
4	\$ E *	$\text{id} / (\text{id} + \text{id}) \$$	Shift	—
5	\$ E * id	$/ (\text{id} + \text{id}) \$$	Reduce	Handle: id ($E \rightarrow \text{id}$)
6	\$ E * E	$/ (\text{id} + \text{id}) \$$	Reduce	Handle: $E * E$ ($E \rightarrow E * E$, left-associative)
7	\$ E	$/ (\text{id} + \text{id}) \$$	Shift	—
8	\$ E /	$(\text{id} + \text{id}) \$$	Shift	—
9	\$ E / (	$\text{id} + \text{id}) \$$	Shift	—
10	\$ E / (id	$+ \text{id}) \$$	Reduce	Handle: id ($E \rightarrow \text{id}$)
11	\$ E / (E	$+ \text{id}) \$$	Shift	—
12	\$ E / (E +	$\text{id}) \$$	Shift	—
13	\$ E / (E + id	$) \$$	Reduce	Handle: id ($E \rightarrow \text{id}$)
14	\$ E / (E + E	$) \$$	Reduce	Handle: $E + E$ ($E \rightarrow E + E$)
15	\$ E / (E	$) \$$	Shift	—

Step	Stack	Remaining Input	Action	Handle Recognized / Notes
16	\$ E / (E)	\$	Reduce	Handle: (E) ($E \rightarrow (E)$)
17	\$ E / E	\$	Reduce	Handle: E / E ($E \rightarrow E/E$)
18	\$ E	\$	ACCEPT	Input successfully parsed

Conflict Resolution Analysis

- **Shift/Reduce Conflict at Step 6 (\$ E * E with input /):**
 - *Conflict:* Should the parser reduce $E * E$ or shift /?
 - *Resolution:* Both * and / share equal precedence and left associativity. The leftmost operator * must be reduced first.
- **Shift/Reduce Conflict inside Parentheses (Step 11 & 14):**
 - Operators inside parentheses take priority over external operators due to higher precedence of ().

Q3.Design and implement a recursive descent parser for the following grammar:

$S \rightarrow (L) \mid a$

$L \rightarrow S L'$

$L' \rightarrow , S L' \mid \epsilon$

Further, demonstrate the parsing process for the input string:

(a)

Your design should:

- Develop recursive procedures for each non-terminal (S, L, and L').
- Apply top-down parsing without backtracking using a predictive recursive descent approach.
- Clearly show the sequence of function calls and terminal-symbol matching during parsing.
- Determine whether the input string is accepted or rejected, with appropriate justification.
- Incorporate a simple error detection and recovery mechanism to handle invalid input strings gracefully.

(10

Marks)

Grammar

$S \rightarrow (L) \mid a$

$L \rightarrow S L'$

$L' \rightarrow , S L' \mid \epsilon$

(i) Recursive Procedures in C-like Pseudocode

C

```
void S() {
    if (lookahead == '(') {
        match('(');
        L();
        match(')');
    } else if (lookahead == 'a') {
        match('a');
    } else {
        error("Syntax Error in S");
    }
}
```

```

void L() {
    S();
    L_prime();
}

void L_prime() {
    if (lookahead == ',') {
        match(',');
        S();
        L_prime();
    } else {
        // epsilon production (do nothing)
        return;
    }
}

void match(char t) {
    if (lookahead == t) {
        lookahead = getNextToken();
    } else {
        error("Mismatch Error");
    }
}

```

(ii) Execution & Function Call Trace for Input (a)

- **Input string:** (a)

Call S()

```

├── match('(') ---> Lookahead becomes 'a'
├── Call L()
│   ├── Call S()
│   │   ├── match('a') ---> Lookahead becomes ')'
│   │   └── Call L_prime()
│   │       └── Lookahead is ')' != ',' ---> Produce ε (return immediately)
│   └── match(')') ---> Lookahead becomes '$' (EOF)

```

Step-by-Step Execution Sequence

Step	Function Active	Lookahead Symbol	Action / Output
1	main()	(	Calls S()
2	S()	(	Matches (→ lookahead updated to a
3	S()	a	Calls L()
4	L()	a	Calls S()
5	S()	a	Matches a → lookahead updated to)
6	L()	)	Calls L_prime()

Step	Function Active	Lookahead Symbol	Action / Output
7	L_prime()	)	Lookahead is not , $\Rightarrow$ returns (ϵ -transition)
8	S()	)	Matches) $\rightarrow$ lookahead updated to \$
9	main()	\$	Parsing completed successfully

(iii) Verification & Error Recovery Mechanism

- **Verification Result:** The input string (a) is **ACCEPTED** as all function calls return cleanly and all terminals match correctly.
- **Error Detection and Recovery Strategy:**
 - **Panic-Mode Recovery:** When a syntax error occurs inside a procedure, discard tokens until a synchronizing token (e.g.,), ,, or \$) is encountered, then resume top-level parsing.

Q4.Design an operator precedence parser for arithmetic expressions involving operators +, -, * and parentheses ().

Further, demonstrate parsing for the input string:

id - id * id

Your design should:

- Construct an unambiguous grammar suitable for operator precedence parsing
- Define precedence relations ($<\cdot$, $=\cdot$, $\cdot>$) among terminals
- Develop the operator precedence table
- Implement stack-based parsing algorithm
- Show step-by-step shift and reduce actions with precedence handling (10 Marks)

Ans:

Target Input String

id – id * id

(i) Unambiguous Grammar Construction

$$E \rightarrow E - T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

(ii) Operator Precedence Relations Matrix

Using operator relations:

- $a <\cdot b$: a yields precedence to b
- $a \cdot> b$: a takes precedence over b
- $a \doteq b$: a has equal precedence to b

Terminal	+	-	*	(	)	id	\$
+	$\cdot>$	$\cdot>$	$<\cdot$	$<\cdot$	$\cdot>$	$<\cdot$	$\cdot>$
-	$\cdot>$	$\cdot>$	$<\cdot$	$<\cdot$	$\cdot>$	$<\cdot$	$\cdot>$
*	$\cdot>$	$\cdot>$	$\cdot>$	$<\cdot$	$\cdot>$	$<\cdot$	$\cdot>$
(	$<\cdot$	$<\cdot$	$<\cdot$	$<\cdot$	$\doteq$	$<\cdot$	Error
)	$\cdot>$	$\cdot>$	$\cdot>$	Error	$\cdot>$	Error	$\cdot>$

Terminal	+	-	*	(	)	id	\$
id	·>	·>	·>	Error	·>	Error	·>
\$	<·	<·	<·	<·	Error	<·	Error

(iii) Step-by-step Operator Precedence Parsing

Input string formatted with relations:

\$\$\$ <\cdot \text{\textit{id}}_1 \cdot> - <\cdot \text{\textit{id}}_2 \cdot> * <\cdot \text{\textit{id}}_3 \cdot> \$\$

Step	Stack	Precedence Relation	Remaining Input	Action
1	\$	<·	id - id * id \$	Shift id
2	\$ id	·>	- id * id \$	Reduce id $\rightarrow E$
3	\$ E	<·	- id * id \$	Shift -
4	\$ E -	<·	id * id \$	Shift id
5	\$ E - id	·>	* id \$	Reduce id $\rightarrow E$
6	\$ E - E	<·	* id \$	Shift * (since $-\prec\cdot$)
7	\$ E - E *	<·	id \$	Shift id
8	\$ E - E * id	·>	\$	Reduce id $\rightarrow E$
9	\$ E - E * E	·>	\$	Reduce $E * E \rightarrow E$ (since $\cdot\prec>$ \$)
10	\$ E - E	·>	\$	Reduce $E - E \rightarrow E$ (since $-\prec\cdot\prec>$ \$)
11	\$ E	—	\$	

Q5.Design and Construct an LALR(1) Parser for the Following Grammar

Consider the grammar:

$S \rightarrow AA$

$A \rightarrow aA \mid b$

A compiler development team is designing an LALR(1) parser for a simple language construct. Before implementing the parser, the team must verify the grammar, construct the required parsing components, and validate the parser using a sample input string.

Demonstrate the operation of the parser for the input string:

bb

Your Design Should:

- Construct the augmented grammar for the given grammar.
- Develop the canonical collection of LR(1) item sets, including closure and goto computations.

- (iii). Compute the FIRST and FOLLOW sets required for parser construction.
- (iv). Merge LR(1) states having identical cores and construct the LALR(1) parsing table (ACTION and GOTO).
- (v). Demonstrate the parsing process for the input string aabb by showing:

Stack contents

Input buffer

Parsing actions (Shift, Reduce, Accept)

Production used during each reduction

- (vi). Verify whether the input string is accepted by the parser and justify the result. (10 Marks)

Ans:

Grammar

$$S \rightarrow AA$$

$$A \rightarrow aA \mid b$$

- **Input string to process:** b b (Note: b b is derived via $S \Rightarrow AA \Rightarrow bA \Rightarrow bb$)

(i) Augmented Grammar

$$(0) S' \rightarrow S$$

$$(1) S \rightarrow AA$$

$$(2) A \rightarrow aA$$

$$(3) A \rightarrow b$$

(ii) FIRST and FOLLOW Sets

- $\text{FIRST}(S) = \{a, b\}$
- $\text{FIRST}(A) = \{a, b\}$
- $\text{FOLLOW}(S) = \{\$ \}$
- $\text{FOLLOW}(A) = \{a, b, \$ \}$

(iii) Canonical LR(1) Items Collection

- I_0 :
 - $S' \rightarrow \cdot S, \$$
 - $S \rightarrow \cdot AA, \$$
 - $A \rightarrow \cdot aA, \{a, b\}$
 - $A \rightarrow \cdot b, \{a, b\}$
- $I_1 = \text{GOTO}(I_0, S)$:
 - $S' \rightarrow S \cdot, \$$
- $I_2 = \text{GOTO}(I_0, A)$:
 - $S \rightarrow A \cdot A, \$$
 - $A \rightarrow \cdot aA, \{\$ \}$
 - $A \rightarrow \cdot b, \{\$ \}$
- $I_3 = \text{GOTO}(I_0, a)$:
 - $A \rightarrow a \cdot A, \{a, b\}$
 - $A \rightarrow \cdot aA, \{a, b\}$
 - $A \rightarrow \cdot b, \{a, b\}$
- $I_4 = \text{GOTO}(I_0, b)$:
 - $A \rightarrow b \cdot, \{a, b\}$
- $I_5 = \text{GOTO}(I_2, A)$:
 - $S \rightarrow AA \cdot, \$$
- $I_6 = \text{GOTO}(I_2, a)$:
 - $A \rightarrow a \cdot A, \{\$ \}$
 - $A \rightarrow \cdot aA, \{\$ \}$
 - $A \rightarrow \cdot b, \{\$ \}$
- $I_7 = \text{GOTO}(I_2, b)$:
 - $A \rightarrow b \cdot, \{\$ \}$

- $I_8 = \text{GOTO}(I_3, A)$:
 - $A \rightarrow aA \cdot, \{a, b\}$
- $I_9 = \text{GOTO}(I_6, A)$:
 - $A \rightarrow aA \cdot, \{\$ \}$

(iv) Merge **LR(1)** Items to Form **LALR(1)** States

Merging states with identical item cores:

- $I_{3,6} = \text{Merge } I_3 \text{ and } I_6$
- $I_{4,7} = \text{Merge } I_4 \text{ and } I_7$
- $I_{8,9} = \text{Merge } I_8 \text{ and } I_9$

LALR(1) Parsing Table

State	Action: a	Action: b	Action: \$	Goto: S	Goto: A
0	$S_{3,6}$	$S_{4,7}$		1	2
1			ACC		
2	$S_{3,6}$	$S_{4,7}$			5
3,6	$S_{3,6}$	$S_{4,7}$			8,9
4,7	r_3	r_3	r_3		
5			r_1		
8,9	r_2	r_2	r_2		

(v) Step-by-Step Parsing Trace for Input b b

- Input: b b \$

Step	Stack	Input Buffer	Parsing Action	Production Used
1	0	b b \$	Shift 4,7	—
2	0 b 4,7	b \$	Reduce 3	$A \rightarrow b$
3	0 A 2	b \$	Shift 4,7	—
4	0 A 2 b 4,7	\$	Reduce 3	$A \rightarrow b$
5	0 A 2 A 5	\$	Reduce 1	$S \rightarrow AA$
6	0 S 1	\$	ACCEPT	$S' \rightarrow S$

(vi) Conclusion and Justification

The input string b b is **successfully ACCEPTED** by the **LALR(1)** parser because:

1. Every state transition follows deterministic entries in the **LALR(1)** parsing table.
2. The sequence of reductions matches the valid derivation: $S \Rightarrow AA \Rightarrow bA \Rightarrow bb$.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	30	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentation & Presentation	10	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation